# Agile vs. Waterfall Process Comparison Report

Software Engineering - SVNC.00.308

Project: StayEase Hotel Reservation System
Team: Cat Bacillus (Kyrylo Pryiomyshev, Danylo Chernov, Daniil Ustiuhov)
Date: March 9, 2026
Version: 1.0

**Project Name:** StayEase Hotel Reservation System

**Team:** Cat Bacillus (Kyrylo Pryiomyshev, Danylo Chernov, Daniil Ustiuhov)

**Date:** March 9, 2026

**Course:** SVNC.00.308 - Software Engineering

## Introduction

In Part II (Waterfall), our team created comprehensive documentation for the StayEase Hotel Reservation System using a sequential, plan-driven approach. We produced a Requirements Specification, Design Documentation, Project Plan with Gantt Chart, Test Plan, Phase Gate Reviews, and a Process Report before writing any code. In Parts III-IV (Agile), we built the same system iteratively across two 90-minute sprints, delivering working features every sprint and adapting based on feedback and retrospective insights. The most striking difference was speed to working software: in Agile, we had a functional booking prototype after Sprint 1 (50 minutes of coding), whereas Waterfall produced only documentation after the same amount of time.

## Comparison Matrix

| Dimension | Waterfall (Part II) | Agile (Parts III-IV) |
|---|---|---|
| Documentation | Created 6 comprehensive documents (Requirements Spec with 12 FRs, Design Doc with architecture and ERD, Gantt Chart, Test Plan with 15 test cases and traceability matrix, Phase Gate Reviews, Process Report). All completed before implementation. | Created lightweight Product Backlog (12 user stories with AC), Sprint Backlogs with task breakdowns, Burndown Charts, and Retrospective Reports. Documents evolved iteratively as we worked and learned. |
| Planning | Planned the entire 11-week project upfront. Created a Gantt Chart with all phases, tasks, dependencies, and milestones. Every task had a start date, end date, and owner defined before any code. | Planned one sprint at a time. Sprint 1 planning took 25 minutes; Sprint 2 used Sprint 1 velocity data. Only the current sprint was planned in detail. Backlog provided a rough roadmap. |
| Feedback | Stakeholder feedback at Phase Gate Reviews (after Requirements and after Design). No feedback on working code, since no code was produced. | Feedback every sprint during Sprint Review. Instructor saw working features and provided input that influenced Sprint 2 priorities. Retrospectives provided internal team feedback. |
| Flexibility | Changing requirements would require updating Requirements Spec, Design Doc, Gantt Chart, Test Plan, and re-approving Phase Gates. Changes were structurally expensive. | Changes expected and welcomed. After Sprint 1, we re-estimated US-05, re-prioritized the backlog, and adjusted Sprint 2 capacity. Adapting felt natural. |
| Risk Discovery | Technical risks identified during design but not validated until implementation. No way to test assumptions without working code. | Discovered US-05 concurrency complexity in Sprint 1 by actually coding it. Real risk surfaced in 90 minutes. Fixed in Sprint 2 with informed approach. |
| Working Code | No working code produced during the Waterfall exercise. All deliverables were | Working code after 50 minutes of Sprint 1. Four features after Sprint 1. Eight features |

| | | |
|---|---|---|
| | planning documents. | after Sprint 2. Incremental delivery every 90 minutes. |
| Collaboration | Document-focused. Each member wrote separate sections. Less frequent integration. Mostly individual, asynchronous work. | Code-focused. Daily standups, parallel task execution, distributed testing. Constant communication about progress and blockers. |

# When to Use Waterfall

## Context 1: Regulated Industries (Medical Devices, Aviation)

- Requirements must be fully documented and approved by regulatory bodies before implementation
- Complete traceability required: every requirement maps to a design element and test case
- Changes require formal re-certification, making them very expensive

Why Waterfall Fits: Our Requirements Specification with traceability matrix and Phase Gate Reviews mirror exactly what regulators require. Waterfall's sequential documentation and formal sign-offs align with compliance audit needs.

## Context 2: Fixed-Price Government Contracts

- Contract specifies exact scope, timeline, and budget before work begins
- Payment milestones tied to phase completion
- Penalties for scope changes or deadline misses

Why Waterfall Fits: Our Gantt Chart provided the detailed timeline and milestones that fixed-price contracts demand. Agile's adaptive planning does not work when the client requires upfront cost and timeline certainty.

## Context 3: Well-Understood Legacy Migration Projects

- Requirements are stable and fully known (replicating existing system functionality)
- Technology is proven and the team has prior experience
- Low uncertainty means iterative discovery adds overhead without proportional benefit

Why Waterfall Fits: When there is nothing to discover, Waterfall's upfront planning is efficient. Agile's exploratory iterations would be wasteful when the solution is already clear.

# When to Use Agile

## Context 1: Startups with Uncertain User Needs

- User needs are unvalidated and the market is uncertain
- Need to pivot quickly based on real user feedback
- Fast time-to-market is critical to stay competitive

Why Agile Fits: Our Sprint Reviews showed working features every 90 minutes. When we re-prioritized our backlog after Sprint 1, we experienced how Agile enables pivots. Waterfall would waste months building features users might not want.

### Context 2: Projects with Active Stakeholder Involvement

- Stakeholder is available and engaged throughout development
- Requirements evolve as the stakeholder sees working features
- Working software matters more than comprehensive documentation

Why Agile Fits: Our sprint demos provided continuous feedback loops. The instructor's feedback after Sprint 1 shaped Sprint 2 priorities. This iterative refinement is only possible with frequent delivery cycles.

### Context 3: High Technical Uncertainty Projects

- Technology or approach is experimental with many unknowns
- Accurate upfront estimation is impossible
- Need to experiment, fail fast, and learn from each iteration

Why Agile Fits: When we discovered US-05's concurrency complexity, we adapted immediately with re-estimation and a targeted solution. Velocity-based planning replaced guesswork with empirical evidence.

## Reflection

Agile felt more engaging and motivating because we saw working code quickly. In Waterfall, we spent hours creating documents without anything executable, which felt abstract. In Agile, within 50 minutes of Sprint 1, we had a working reservation system. Demonstrating real functionality at Sprint Review provided a sense of accomplishment that Phase Gate document reviews did not match. The retrospective cycle was particularly impactful: Sprint 1 mistakes directly became Sprint 2 improvements.

However, Waterfall gave us a clearer big-picture view. The Gantt Chart showed how all phases fit together, and the Design Documentation forced us to think through the complete architecture before coding. In Agile, we only planned one sprint at a time, which occasionally made it harder to see the overall product vision. If we were building StayEase for a real hotel with regulatory requirements, Waterfall's compliance documentation would be essential. For our class project with evolving requirements and a small team, Agile was the clearly better fit. The key insight is that neither methodology is universally superior — choosing the right approach depends on the project's constraints, risks, and stakeholder needs.

## Team Sign-Off

Kyrylo Pryiomyshev: _____ Date: March 9, 2026

Danylo Chernov: _____ Date: March 9, 2026

Daniil Ustiuhov: _____ Date: March 9, 2026